

TUGETHA

System Design & Product Blueprint - Version 2.0

Save Together. Lend to Friends. Achieve Goals.

Version	2.0.0	Status	Active Development
Mobile	Flutter (Android-first, iOS later)	Backend	Java 21 + Spring Boot 3.x
Database	PostgreSQL + Flyway	Cache	Redis
Payments	Paystack -> Daraja -> Bank API	Auth	Firebase Auth (OTP) -> Custom OTP V3
Built by	Sinaps Technology	Contact	synapsetechnology14@gmail.com

CONFIDENTIAL - For internal use and investor review only

Version History

Version	Date	Changes
v1.0	Jun 2026	Initial blueprint - Flutter + Firebase + Django Python payment backend
v2.0	Jun 2026	Full stack upgrade: Django -> Java 21 + Spring Boot 3.x. Full Firebase -> PostgreSQL migration plan. Payment abstraction layer introduced. Quartz scheduler, RabbitMQ roadmap, React/Next.js admin dashboard. Complete system redesign for production-grade financial system.

1. Executive Summary

Tugetha is a Flutter-based community finance mobile application built by Sinaps Technology. It digitizes Kenyan communal financial practices - chamas, harambees, peer-to-peer lending, and shared goal savings - into a structured, transparent, and secure platform.

Version 2.0 of this blueprint reflects a deliberate architectural upgrade. The original Python/Django payment backend has been replaced with Java 21 + Spring Boot 3.x - a decision driven by production-grade reliability requirements for a financial system. Firebase, which served well for rapid MVP development, is now positioned as a transitional layer with a structured migration path to the Spring Boot backend as the single source of truth for all data.

Core philosophy: Tugetha is not a social app that happens to handle money. It is a financial ledger product that happens to have a social interface. Every architectural decision prioritizes money safety first, user experience second.

App name	Tugetha
Platform	Flutter - Android-first, iOS support planned
Backend	Java 21 + Spring Boot 3.x
Primary database	PostgreSQL with Flyway migrations
Cache / locks	Redis
Payments	Paystack (live) -> Daraja (V2) -> Bank API (V3)
Auth (current)	Firebase Auth - phone OTP
Auth (target)	Custom OTP via Africa's Talking - V3
Notifications	Firebase FCM (push) + Africa's Talking (SMS)
Admin dashboard	React / Next.js + Tailwind CSS
Monitoring	Sentry + Prometheus + Grafana
Infrastructure	Docker + GitHub Actions + managed PostgreSQL on Render
Target market	Kenya - East Africa expansion V3
Monetization	Transaction fees per financial action

2. Technology Stack (v2.0)

The following is the complete, committed technology stack for Tugetha. Every component was chosen for a specific reason - this section documents those reasons.

2.1 Mobile App - Flutter

Framework	Flutter (Dart) - cross-platform Android and iOS
State management	Riverpod - reactive, testable, compile-safe
Navigation	GoRouter - declarative routing with deep link support
HTTP client	Dio with Retrofit - type-safe API calls, interceptors for JWT
Push notifications	Firebase Cloud Messaging - integrates with Spring Boot notification module
Biometrics	local_auth - fingerprint and face ID for V2
Secure storage	flutter_secure_storage - JWT token, PIN hash stored securely on device
Offline support	SQLite via sqflite - cache critical data for poor connectivity

Design rule: Flutter handles UI, validation, auth token storage, and API calls only. Zero business logic, zero Paystack keys, zero financial calculation in the Flutter app. All logic lives in Spring Boot.

2.2 Backend Core - Java 21 + Spring Boot 3.x

Spring Boot 3.x on Java 21 is the brain of Tugetha. It is the single source of truth for all financial data and owns every money movement in the system.

Runtime	Java 21 LTS - virtual threads (Project Loom) for high concurrency
Framework	Spring Boot 3.x - battle-tested, extensive ecosystem, production-proven in fintech
Web layer	Spring Web (REST) - controllers, filters, exception handlers
Security	Spring Security - JWT filter chain, Firebase token verification, role-based access
Data access	Spring Data JPA + Hibernate - type-safe repository pattern
Database migration	Flyway - versioned, auditable schema changes. Never manual DB edits
Validation	Jakarta Bean Validation - request validation at controller layer
Background jobs	Spring Scheduler (MVP) -> Quartz (production) -> RabbitMQ (event-driven)

Testing	JUnit 5 + Mockito + Testcontainers - unit, integration, and contract tests
----------------	--

Why Java Spring Boot over Python Django?

Criterion	Python Django	Java Spring Boot (chosen)
Type safety	Dynamic typing - runtime errors possible	Static typing - financial bugs caught at compile time
Concurrency	GIL limits true parallelism	Virtual threads (Java 21) - high concurrency natively
Financial ecosystem	Limited fintech libraries	Mature fintech, banking, and audit libraries
Performance	Adequate for MVP	Significantly higher throughput for payment processing
Production track record	Good for web apps	Powers major banks and payment processors globally
Long-term migration	Would need rewrite anyway	Scales from startup to enterprise without rewrite
Team skill	Ric familiar with Django	Spring Boot is standard for serious backend systems

2.3 Database Layer

PostgreSQL - Primary Database

PostgreSQL is the source of truth for everything important in Tugetha. Every schema change is version-controlled via Flyway. No manual production database edits - ever.

Complete table list:

- users, user_profiles, user_devices
- wallet_accounts, ledger_accounts
- ledger_entries (IMMUTABLE), transactions, payment_attempts
- webhook_events (PERMANENT LOG)
- loans, loan_repayments
- groups, group_members
- goals, goal_contributions
- chamas, chama_cycles, chama_contributions
- standing_orders (consent-based auto-deductions)
- notifications, notification_preferences
- kyc_records

- audit_logs (append-only)
- idempotency_keys

Flyway - Database Migration Tool

Every schema change is a numbered SQL file in version control (V001__create_users.sql, V002__add_wallet_table.sql). No migration = no change. This gives a complete, auditable history of every structural change ever made to the database.

Redis - Cache and Distributed Locks

Idempotency keys	7-day TTL - prevents double-processing payments under any network condition
Wallet balance cache	Short TTL cache of computed balances - reduces ledger aggregate queries
Distributed locks	Prevents concurrent processing of same transaction across multiple server instances
Session cache	JWT token blacklist for logout
Rate limiting	Per-user API rate limiting for payment endpoints

2.4 Payment Abstraction Layer

The most important architectural decision in the payments module: never hardcode Paystack everywhere. A payment provider abstraction interface means the payment rail can be switched or extended without rewriting business logic.

```
PaymentProvider (interface)
+-- initializeCollection(amount, currency, metadata) -> PaymentInit
+-- verifyPayment(reference) -> PaymentVerification
+-- initializeTransfer(amount, recipient, reason) -> TransferInit
+-- verifyTransfer(reference) -> TransferVerification
+-- createRecipient(name, phone, type) -> RecipientCode
+-- handleWebhookEvent(payload, signature) -> WebhookResult

PaystackProvider implements PaymentProvider <- live now
DarajaProvider   implements PaymentProvider <- V2 roadmap
BankProvider     implements PaymentProvider <- V3 roadmap
```

Business logic (ledger, wallet, loans) calls PaymentProvider - it never knows or cares whether Paystack or Daraja executed the actual transaction. This is the abstraction that future-proofs the entire payment system.

2.5 Background Jobs

Spring Scheduler (MVP)	Simple cron-based jobs - sufficient for initial launch volume
Quartz (Production)	Clustered job scheduling - survives server restarts, supports job history
RabbitMQ (V2)	Event-driven message queue - decouple services, handle payment event streams

Jobs required:

- Payment verification retry - for transactions stuck in INITIATED state
- Stuck transaction recovery - 5-minute sweep, calls Paystack verify API
- Daily reconciliation - total debits must equal total credits
- Loan repayment reminders - push + SMS 3 days and 1 day before due date
- Auto-deduction (standing orders) - only for consented members, checks balance first
- Chama contribution reminders - per-cycle schedule
- Failed withdrawal recovery - reverse ledger on transfer_failed event
- Weekly user summary - wallet, loans, goals digest every Monday
- Trust score recalculation - nightly batch based on repayment history

2.6 Authentication Strategy

Current (V1-V2): Firebase Auth

OTP delivery	Firebase sends SMS to +254 numbers - no Africa's Talking account needed yet
Token format	Firebase ID token (JWT) - sent in Authorization header to Spring Boot
Spring Boot verification	Firebase Admin SDK verifies token - extracts UID, phone number
User mapping	Firebase UID stored in users.firebase_uid - links Firebase and PostgreSQL records

Target (V3): Custom OTP + Spring Security JWT

OTP delivery	Africa's Talking SMS - direct control, lower cost at scale, no vendor dependency
Token format	Spring Boot issues its own JWT - no Firebase dependency for auth
Benefits	Full control of session lifecycle, custom expiry, refresh token rotation
Migration path	Gradual - new users get custom auth, existing Firebase users migrated progressively

2.7 Notifications

Push (app is open/background)	Firestore Cloud Messaging - Spring Boot calls FCM HTTP API
SMS (critical financial events)	Africa's Talking SMS API - withdrawal, loan overdue, standing order, security alerts
Email (future)	SendGrid or Resend - account statements, KYC results, admin alerts
In-app notification center	Stored in PostgreSQL notifications table - read/unread state, history
Notification preferences	Per-user opt-in/out per channel - stored in PostgreSQL

2.8 Admin Dashboard

A fintech app without admin tooling is blind. The admin dashboard is a non-negotiable V1.5 deliverable.

Frontend	React / Next.js + Tailwind CSS
API	Spring Boot admin endpoints - role-protected (ADMIN role required)
User management	User lookup, KYC review and approval, account suspension
Transaction audit	Search by reference, user, amount, date range, status
Wallet audit	View any user wallet ledger - all entries, all time
Failed payments	List of stuck/failed transactions with retry capability
Manual reconciliation	Trigger reconciliation run, view discrepancy reports
Loan monitoring	Active loans, overdue loans, default flags, manual interventions
Chama monitoring	Cycle status, pending disbursements, contribution gaps
Webhook logs	Full raw log of every inbound webhook - search by reference
System health	Database connection, Redis, background job status, API latency

2.9 Monitoring & Observability

Error tracking	Sentry - real-time error alerts, stack traces, breadcrumbs
Metrics	Prometheus - JVM metrics, custom payment metrics, latency histograms
Dashboards	Grafana - payment success rate, ledger balance, active users, error rate
Logging	Structured JSON logs - searchable in production, correlation IDs per request
Health checks	Spring Boot Actuator - /health, /metrics, /info endpoints

Alerts

PagerDuty or simple email - critical alerts for ledger imbalance, high error rate

3. Firebase to Spring Boot Migration Plan

Firebase served Tugetha well during rapid MVP development. The migration to a fully custom Spring Boot backend is not a big-bang rewrite - it is a gradual, feature-by-feature migration that maintains zero downtime and preserves all existing user data.

Migration principle: At any point during the migration, the app is fully functional. We move one data type at a time, validate, then move the next. Firebase is never "switched off" - it is progressively replaced.

3.1 Migration Phases

Phase	Timeline	What moves
Phase 0 Current state	Now	Firebase Auth + Firestore holds all data. Django Python handles payments (being replaced by Spring Boot). Flutter reads/writes Firestore directly for all app data.
Phase 1 Payment backend	Weeks 1-4	Replace Django with Spring Boot for all payment operations. Firestore remains for app data. Firebase Auth remains. Spring Boot syncs wallet balance to Firestore for display.
Phase 2 User & wallet data	Weeks 5-8	Users, profiles, wallet history migrate to PostgreSQL. Spring Boot becomes source of truth for user data. Flutter calls Spring Boot APIs for user operations. Firestore used as cache only for these.
Phase 3 Groups & goals	Weeks 9-12	Groups, goals, contributions migrate to PostgreSQL. Spring Boot exposes Groups and Goals APIs. Firestore real-time listeners replaced with Spring Boot Server-Sent Events or polling.
Phase 4 Loans	Weeks 13-16	Loan metadata migrates to PostgreSQL (ledger entries already there). Spring Boot becomes source of truth for all loan data. Firestore loan records deprecated.
Phase 5 Chama	Weeks 17-20	Full Chama module in Spring Boot - cycles, contributions, standing orders. WhatsApp integration wired to Spring Boot webhook endpoint.
Phase 6 Auth migration	V3	Replace Firebase Auth OTP with Africa's Talking SMS + Spring Boot JWT. Existing Firebase users migrated progressively. Firebase dependency removed entirely.

3.2 Data Migration Strategy

For each Firestore collection being migrated:

1. Write Spring Boot API for the new data type - fully tested
2. Run dual-write: Flutter writes to both Firestore AND Spring Boot API simultaneously
3. Validate: compare Firestore and PostgreSQL records - ensure consistency
4. Switch reads: Flutter reads from Spring Boot API only

5. Deprecate Firestore writes: remove Firestore writes from Flutter
6. Archive Firestore data: export to cold storage for audit trail

Dual-write period typically lasts 2-4 weeks per data type to catch edge cases before fully cutting over.

3.3 What Firebase Keeps (Permanently)

Firebase Auth (V1-V2)	Phone OTP delivery - replaced by Africa's Talking only in V3
Firebase FCM	Push notification delivery - no equivalent self-hosted solution needed
Firebase Storage	KYC document storage - private bucket, suitable for document storage

4. Spring Boot Module Design

The Spring Boot backend is organized into focused modules, each owning a specific business domain. Modules communicate through well-defined service interfaces - never directly accessing each other's repositories.

4.1 Project Structure

```
tugetha-backend/
+-- src/main/java/tech/sinaps/tugetha/
|   +-- TugethaApplication.java
|   |
|   +-- config/
|   |   +-- SecurityConfig.java      # Spring Security filter chain
|   |   +-- FirebaseConfig.java     # Firebase Admin SDK init
|   |   +-- RedisConfig.java
|   |   +-- QuartzConfig.java
|   |
|   +-- common/
|   |   +-- auth/                   # Firebase JWT filter
|   |   +-- idempotency/            # Idempotency key guard
|   |   +-- exception/             # Global exception handler
|   |   +-- firebase/              # Firestore sync service
|   |   +-- audit/                 # Audit log writer
|   |
|   +-- modules/
|   |   +-- user/
|   |   +-- wallet/
|   |   +-- ledger/                # Double-entry bookkeeping CORE
|   |   +-- payment/              # Paystack / payment provider
|   |   +-- loan/
|   |   +-- group/
|   |   +-- goal/
|   |   +-- chama/
|   |   +-- notification/
|   |   +-- webhook/
|   |   +-- kyc/
|   |   +-- admin/
|   |
|   +-- scheduler/
|   |   +-- PaymentRecoveryJob.java
|   |   +-- ReconciliationJob.java
|   |   +-- LoanReminderJob.java
|   |   +-- ChamaContributionJob.java
|   |
|   +-- src/main/resources/
|   |   +-- db/migration/
|   |   |   +-- V001__create_users.sql
|   |   |   +-- V002__create_ledger.sql
|   |   |   +-- V003__create_loans.sql
|   |   +-- application.yml
|   |
|   +-- src/test/                  # JUnit 5 + Testcontainers
|   +-- Dockerfile
|   +-- docker-compose.yml
+-- pom.xml
```

4.2 Ledger Module (Most Critical)

The ledger module is the financial core. It enforces double-entry bookkeeping, immutability, and the transaction state machine. Every other module that moves money calls the ledger service - it never writes directly to ledger tables.

Core entities:

```
@Entity
public class Transaction {
    UUID id; // primary key
    TransactionType type; // TOPUP, WITHDRAWAL, LOAN_DISBURSEMENT, etc.
    TransactionStatus status; // state machine enforced in service layer
    String idempotencyKey; // UNIQUE - prevents double processing
    BigDecimal amount;
    String currency; // KES
    User initiatedBy;
    String firebaseUid;
    String paystackReference; // UNIQUE NULLABLE
    String failureReason;
    List<StateTransition> stateHistory; // append-only
    Map<String, Object> metadata;
    Instant createdAt;
    Instant updatedAt;
    Instant completedAt;
}

@Entity
public class LedgerEntry {
    UUID id;
    Transaction transaction;
    EntryType entryType; // DEBIT | CREDIT
    AccountType accountType; // USER_WALLET | PLATFORM_FEE | etc.
    User user; // null for platform accounts
    BigDecimal amount; // always positive
    String description;
    String reference; // UNIQUE - "{txId}-debit-wallet"
    Instant createdAt; // NEVER updated
}
```

4.3 Payment Module

The payment module orchestrates the Paystack integration via the PaymentProvider abstraction. It never posts ledger entries directly - it calls LedgerService.

PaymentProvider interface	Abstraction over Paystack, Daraja, and future providers
PaystackProvider	Implements PaymentProvider - all Paystack HTTP calls
TopUpService	Initiate -> verify -> post ledger -> sync Firestore

WithdrawalService	Balance check -> reserve funds -> transfer -> webhook confirms
PaymentRecoveryService	Finds stuck transactions, retries verification with Paystack
IdempotencyService	Redis-backed guard - check_and_lock, mark_complete, mark_failed

4.4 Chama Module (V2 - Digital Merry-Go-Round)

The chama module is the most complex feature in Tugetha. It manages the full merry-go-round lifecycle with WhatsApp payment monitoring, optional standing orders, and automatic disbursement.

ChamaGroup entity	Group metadata, contribution amount, cycle type, member list, order
ChamaCycle entity	One cycle per rotation - tracks which member receives, cycle dates, status
ChamaContribution entity	Individual payment record per member per cycle - linked to ledger transaction
StandingOrder entity	Consent record per member - explicit opt-in, cancellable at any time
ChamaService	Core lifecycle: create, add member, start cycle, record contribution, disburse
WhatsAppWebhookHandler	Parses payment confirmation messages, calls ChamaService.recordContribution()
DarajaWebhookHandler	C2B payment webhook - auto-records contribution when till/paybill receives payment
ChamaDisbursementService	Atomic: verify all paid -> post ledger -> initiate Paystack transfer to receiver
StandingOrderJob	Nightly: check due dates -> verify consent -> verify balance -> deduct or skip

Standing order design: The job checks standing_orders table for due records. If consented=true AND balance >= amount: deduct and notify. If balance < amount: send reminder, do NOT deduct. The member is never surprised.

4.5 API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/payments/topup/initiate	Initialize Paystack checkout - returns payment URL
POST	/api/v1/payments/topup/verify	Verify payment + post ledger + sync Firestore

POST	/api/v1/payments/withdraw	Initiate M-Pesa withdrawal via Paystack transfer
GET	/api/v1/wallet/balance	Authoritative balance from ledger
GET	/api/v1/wallet/transactions	Transaction history - last 50
POST	/api/v1/loans/request	Create loan request
POST	/api/v1/loans/{id}/disburse	Approve and disburse loan atomically
POST	/api/v1/loans/{id}/repay	Process repayment - debit borrower, credit lender
POST	/api/v1/goals/{id}/contribute	Deduct wallet + record goal contribution
POST	/api/v1/chama/contribution	Record chama contribution (WhatsApp or manual)
POST	/api/v1/chama/{id}/disburse	Trigger disbursement when all contributions received
POST	/api/v1/standing-orders	Create standing order - requires explicit consent + PIN
DELETE	/api/v1/standing-orders/{id}	Cancel standing order - effective immediately
POST	/api/v1/webhooks/paystack	Inbound Paystack webhook - HMAC verified
POST	/api/v1/webhooks/whatsapp	Inbound WhatsApp payment notification
GET	/api/v1/ledger/reconcile	Admin: run reconciliation check
GET	/actuator/health	Service health - no auth required

5. Financial System Design

The financial subsystem is unchanged from v1.0 in philosophy but now implemented in Java with Spring's `@Transactional` annotation ensuring atomicity at the database level. Every principle is the same - immutability, idempotency, double-entry, state machine, failure recovery.

5.1 Double-Entry Bookkeeping

All money movements create exactly two ledger entries. Total debits = total credits always. Daily reconciliation job verifies this invariant.

```
Top up KES 1,000:
DEBIT  PAYSTACK_COLLECTION  +1,000  money received from Paystack
CREDIT USER_WALLET          +1,000  user balance increases

Withdrawal KES 500 + KES 10 fee:
DEBIT  USER_WALLET          +510    user balance decreases
CREDIT PAYSTACK_PAYOUT      +500    money going out
CREDIT PLATFORM_FEE         +10     Tugetha earns KES 10

Loan disbursement KES 2,000 (2.5% fee = KES 50):
DEBIT  USER_WALLET (lender) +2,000  lender balance decreases
CREDIT USER_WALLET (borrower)+1,950 borrower receives net amount
CREDIT PLATFORM_FEE         +50     Tugetha earns 2.5%

Chama disbursement KES 5,000 (1% fee = KES 50):
DEBIT  PLATFORM_ESCROW      +5,000  escrow released
CREDIT USER_WALLET (receiver)+4,950 receiver gets net amount
CREDIT PLATFORM_FEE         +50     Tugetha earns 1%
```

5.2 Wallet Balance Computation

Wallet balance is never stored as a single editable number. It is always computed from the ledger - this is the invariant that makes it impossible to lose funds through direct database manipulation.

```
// LedgerService.java
public BigDecimal getBalance(User user) {
    BigDecimal credits = ledgerEntryRepo
        .sumByUserAndAccountTypeAndEntryType(
            user, USER_WALLET, CREDIT
        ).orElse(BigDecimal.ZERO);

    BigDecimal debits = ledgerEntryRepo
        .sumByUserAndAccountTypeAndEntryType(
            user, USER_WALLET, DEBIT
        ).orElse(BigDecimal.ZERO);

    return credits.subtract(debits);
    // Result cached in Redis for 30 seconds
```



```

    // Cache invalidated on every new ledger entry for this user
}

```

5.3 Transaction State Machine

```

INITIATED --> PROCESSING --> COMPLETED
|           |           |
+--> FAILED   +--> FAILED   +--> REVERSED

public enum TransactionStatus {
    INITIATED, PROCESSING, COMPLETED, FAILED, REVERSED;

    private static final Map<TransactionStatus, Set<TransactionStatus>>
        VALID_TRANSITIONS = Map.of(
            INITIATED, Set.of(PROCESSING, COMPLETED, FAILED),
            PROCESSING, Set.of(COMPLETED, FAILED),
            COMPLETED, Set.of(REVERSED),
            FAILED, Set.of(),
            REVERSED, Set.of()
        );

    public void validateTransition(TransactionStatus next) {
        if (!VALID_TRANSITIONS.get(this).contains(next)) {
            throw new InvalidStateTransitionException(this, next);
        }
    }
}

```

6. Failure Modes & Recovery

Every failure mode is designed with a specific recovery path. The system must never leave funds in an unknown state.

Scenario 1 - User pays Paystack, webhook never arrives

Flutter calls `/api/v1/payments/topup/verify/` immediately after Paystack checkout returns - this is the primary confirmation path. Webhook is a secondary confirmation.

Recovery: `PaymentRecoveryJob` runs every 5 minutes. Finds INITIATED transactions > 10 minutes old. Calls Paystack verify API. Posts ledger if confirmed. Marks FAILED after 24 hours. Admin alerted via Sentry.

Scenario 2 - Withdrawal transfer fails

@Transactional wraps: balance check -> ledger DEBIT -> Paystack transfer. If Paystack fails, Spring rolls back the entire transaction - no money moved.

Recovery: On `transfer.failed` webhook -> post reversal ledger entries atomically. User wallet instantly restored. Push + SMS: "Withdrawal failed - your KES X is safe in your wallet." Sentry alert fired.

Scenario 3 - Server crash mid-payment

Idempotency key prevents double-processing on restart. Quartz job state persists in PostgreSQL - survives crash. On restart, job resumes and completes or fails cleanly.

Recovery: `PaymentRecoveryJob` finds stuck transactions and resolves them. Idempotency key stored in PostgreSQL (not Redis alone) - survives Redis restart too.

Scenario 4 - Duplicate Paystack webhook

`WebhookEvent` table stores every raw webhook. `paystack_reference` has UNIQUE constraint. `IdempotencyService` checks before processing. Second webhook for same reference: log it, return 200, do nothing.

Scenario 5 - Chama standing order, insufficient balance

Standing order job checks balance before ANY deduction. If balance < contribution amount: send reminder notification, record as SKIPPED, do NOT deduct. Member must contribute manually. No overdraft possible.

The standing order is consent-based and skip-safe. It never causes a negative wallet balance. The member is notified clearly: "Your standing order for Chama X could not be processed - insufficient balance."

7. Infrastructure & Deployment

7.1 MVP Infrastructure (Render)

Spring Boot API	Render Web Service - auto-deploy from GitHub main branch
PostgreSQL	Render managed PostgreSQL - automated backups, connection pooling
Redis	Render managed Redis - used for idempotency, cache, rate limiting
Background worker	Render background worker - runs Quartz jobs separately from API
Admin dashboard	Vercel - Next.js admin app, separate deployment
Domain & SSL	Cloudflare - DNS, SSL termination, DDoS protection
CI/CD	GitHub Actions - build, test, migrate, deploy on every push to main

7.2 Production Infrastructure

Render free tier is not suitable for real money movement at scale. When Tugetha reaches consistent revenue, migrate to dedicated cloud infrastructure.

Compute	VPS or cloud (DigitalOcean / AWS) - Docker containers via Docker Compose or Kubernetes
Database	Dedicated PostgreSQL with read replica for analytics queries
Redis	Redis Cluster - high availability, persistence enabled
Load balancer	Nginx reverse proxy - SSL termination, rate limiting, static file serving
Monitoring	Prometheus + Grafana dashboards, Sentry error tracking
Backups	Automated daily PostgreSQL backups - 30-day retention, tested monthly restore
Security	WAF, intrusion detection, automated vulnerability scanning
Compliance	Kenya Data Protection Act compliance review - user data, financial records

7.3 GitHub Actions CI/CD Pipeline

```
On push to main:
1. Run unit tests (JUnit 5)
2. Run integration tests (Testcontainers - real PostgreSQL + Redis)
3. Run Flyway migration on test database
4. Build Docker image
5. Push to container registry
```

6. Deploy to Render (staging)
7. Run smoke tests on staging
8. Promote to production (manual approval gate)
9. Run Flyway migration on production
10. Health check - confirm /actuator/health returns 200

8. Updated Feature Breakdown (v2.0)

All features from v1.0 are retained. This section documents new features added in v2.0 and clarifications to existing features based on architecture decisions.

8.1 Digital Merry-Go-Round Chama (New - V2)

The MGR chama is the most impactful feature for the Kenyan market. Full specification updated to reflect Spring Boot implementation and WhatsApp integration design.

Chama type	Merry-go-round - fixed contribution, rotating recipient
Contribution amount	Fixed per member - set at chama creation, unchangeable mid-cycle
Cycle frequency	Weekly, bi-weekly, or monthly - set at creation
Member order	Manual (admin sets order) or randomized by app at cycle start
WhatsApp integration	Bot monitors chama WhatsApp group - parses M-Pesa confirmation SMS shared in group
Till/Paybill detection	Daraja C2B webhook - payment to registered till auto-records contribution
Manual confirmation	Admin can manually confirm a contribution if auto-detection fails
Disbursement trigger	Automatic when all members for current cycle have paid
Disbursement method	Paystack Transfer API to receiver's registered M-Pesa number
Standing order	OPTIONAL per member - requires explicit consent screen + PIN + can cancel anytime
Auto-deduction rule	Only if: consented=true AND balance >= contribution amount on due date
Skip behaviour	If balance insufficient - reminder sent, contribution skipped, no deduction
1% disbursement fee	Charged to receiver on auto-disbursement - shown clearly at setup

8.2 Standing Order Design (Detailed)

The standing order is one of the most sensitive features in Tugetha. It must be designed with maximum user consent and transparency to maintain trust.

Setup flow:

7. Member opens chama settings -> taps "Set up standing order"
8. Consent screen shown: exact amount, exact date, what happens if balance is low, how to cancel
9. Member reads and taps "I understand and agree"

10. Member enters transaction PIN to confirm
11. StandingOrder record created in PostgreSQL with consent_timestamp and consent_ip
12. Confirmation push notification sent: "Standing order set up for Chama X - KES Y on the Nth of each month"

Cancellation flow:

13. Member opens chama settings -> taps "Cancel standing order"
14. Confirmation dialog: "Your standing order will be cancelled. You will need to contribute manually."
15. Member confirms
16. StandingOrder.active = false - effective immediately
17. Push notification: "Standing order cancelled for Chama X"

Cancellation is always allowed, always instant, always zero friction. A standing order that is hard to cancel is a standing order that destroys trust.

8.3 Trust Score Engine (V2 - Updated Design)

Score range	0-100 - all new users start at 50
Positive factors	On-time loan repayment (+5), early repayment (+8), wallet top-up activity (+1), chama consistency (+3)
Negative factors	Late repayment (-5 per week late), default (-20), chama skip (-3), missed standing order (-1)
Score visibility	User sees their own score. Can share via link to potential lenders.
Score impact	Higher score unlocks higher loan request limits. Shown to lenders when reviewing request.
Recalculation	Nightly batch job - all active users recalculated
CRB reporting	Not in V1-V2. V3: opt-in submission to Kenya CRBs with explicit user consent

9. Database Schema (PostgreSQL)

Key tables in the PostgreSQL database. All tables include standard audit fields: `created_at`, `updated_at`. Financial tables additionally include version (optimistic locking) and `audit_log_id`.

users

```
id          BIGSERIAL      PRIMARY KEY
firebase_uid VARCHAR(128)  UNIQUE NOT NULL -- mapped to Firebase Auth
phone       VARCHAR(20)  UNIQUE NOT NULL
name        VARCHAR(255) NOT NULL
mpesa_number VARCHAR(20)
email       VARCHAR(255)
trust_score INT          DEFAULT 50
kyc_status  VARCHAR(20)  DEFAULT 'PENDING'
role        VARCHAR(20)  DEFAULT 'USER' -- USER | ADMIN
active      BOOLEAN      DEFAULT TRUE
created_at  TIMESTAMPTZ  DEFAULT NOW()
updated_at  TIMESTAMPTZ
```

transactions

```
id          UUID          PRIMARY KEY DEFAULT gen_random_uuid()
transaction_type VARCHAR(30)  NOT NULL
status      VARCHAR(20)  NOT NULL DEFAULT 'INITIATED'
idempotency_key VARCHAR(255) UNIQUE NOT NULL
amount      NUMERIC(12,2) NOT NULL CHECK (amount > 0)
currency    VARCHAR(3)   DEFAULT 'KES'
initiated_by BIGINT       REFERENCES users(id)
firebase_uid VARCHAR(128)  NOT NULL
paystack_reference VARCHAR(255) UNIQUE
failure_reason TEXT
state_history JSONB         DEFAULT '[]'
metadata     JSONB         DEFAULT '{} '
created_at   TIMESTAMPTZ  DEFAULT NOW()
updated_at   TIMESTAMPTZ
completed_at TIMESTAMPTZ

INDEX idx_transactions_firebase_uid_status (firebase_uid, status)
INDEX idx_transactions_status_created (status, created_at)
INDEX idx_transactions_paystack_ref (paystack_reference)
```

ledger_entries (IMMUTABLE - no UPDATE, no DELETE)

```
id          UUID          PRIMARY KEY DEFAULT gen_random_uuid()
transaction_id UUID        NOT NULL REFERENCES transactions(id)
entry_type   VARCHAR(6)   NOT NULL CHECK (entry_type IN ('DEBIT', 'CREDIT'))
account_type VARCHAR(30)  NOT NULL
user_id      BIGINT       REFERENCES users(id)
amount      NUMERIC(12,2) NOT NULL CHECK (amount > 0)
```

```

currency      VARCHAR(3)    DEFAULT 'KES'
description   TEXT        NOT NULL
reference     VARCHAR(255) UNIQUE NOT NULL
created_at    TIMESTAMPTZ DEFAULT NOW() -- immutable, set once
metadata      JSONB       DEFAULT '{}'
```

```

-- Trigger prevents UPDATE and DELETE:
CREATE OR REPLACE RULE no_update_ledger AS ON UPDATE TO ledger_entries
DO INSTEAD NOTHING;
CREATE OR REPLACE RULE no_delete_ledger AS ON DELETE FROM ledger_entries
DO INSTEAD NOTHING;
```

standing_orders

```

id            BIGSERIAL  PRIMARY KEY
user_id       BIGINT     REFERENCES users(id)
chama_id      BIGINT     REFERENCES chamas(id)
amount        NUMERIC(12,2)
deduction_day INT        CHECK (deduction_day BETWEEN 1 AND 31)
active        BOOLEAN    DEFAULT TRUE
consent_timestamp TIMESTAMPTZ NOT NULL
consent_ip    VARCHAR(45)
cancelled_at  TIMESTAMPTZ
cancel_reason TEXT
created_at    TIMESTAMPTZ DEFAULT NOW()
```


10. Development Roadmap (v2.0)

Phase	Timeline	Deliverables
Phase 1 Spring Boot foundation	Weeks 1-3	Project setup (Spring Boot 3.x, Java 21, PostgreSQL, Flyway), common module (Firebase auth filter, idempotency, exception handler), ledger module (Transaction, LedgerEntry, LedgerService), health endpoint, unit tests
Phase 2 Payment module	Weeks 4-6	PaymentProvider abstraction, PaystackProvider implementation, TopUpService, WithdrawalService, webhook handler, PaymentRecoveryJob, Firestore sync service, integration tests with Testcontainers
Phase 3 Loan module	Weeks 7-8	Loan entity, LoanService (request, approve, disburse, repay), loan disbursement via LedgerService atomically, repayment scheduler, Firestore loan sync, Flutter updated to call Spring Boot loans API
Phase 4 Goal & Group modules	Weeks 9-10	Group and Goal entities, contribution endpoint wired to LedgerService, Firestore sync, Flutter updated to call Spring Boot for goals
Phase 5 Chama module	Weeks 11-14	ChamaGroup, ChamaCycle, ChamaContribution entities, WhatsApp webhook handler, Daraja C2B webhook handler, StandingOrder with consent flow, auto-disbursement, ChamaContributionJob
Phase 6 Admin dashboard	Weeks 15-17	React/Next.js admin app, user management, transaction search, wallet audit, webhook logs, manual reconciliation, KYC review
Phase 7 Monitoring & hardening	Weeks 18-20	Sentry integration, Prometheus metrics, Grafana dashboards, load testing, security audit, penetration testing basics, Play Store submission
Firebase migration	Month 5-8	Gradual dual-write migration: users -> groups -> goals -> loans -> full Spring Boot as source of truth
V3 - Custom auth	Month 9-12	Africa's Talking OTP, Spring Security JWT, Firebase Auth removed, full independence from Firebase

11. Risks & Mitigations (v2.0)

Risk	Severity	Mitigation
Spring Boot learning curve vs Django familiarity	Medium	Ric has strong backend fundamentals - Spring Boot patterns map well to Django. Jhub Africa network for peer review. Start with simpler modules before ledger.
Firebase migration disruption	High	Dual-write strategy - no big-bang cutover. Each collection migrated independently with validation period. Rollback possible at any phase.
CBK regulation for lending	High	Launch as social savings/community finance. Consult fintech lawyer before enabling interest on loans. Monitor CBK DCP licensing. MGR chama framed as community savings not lending.
Standing order user trust	High	Maximum consent friction at setup. Zero friction at cancellation. Skip-safe (never overdraft). Full audit trail. Clear notifications before every deduction.
Paystack Kenya M-Pesa support	Medium	Test with Paystack Kenya sandbox - confirm M-Pesa transfer API fully supported. Daraja integration in V2 as backup rail if Paystack transfer reliability insufficient.
Render free tier reliability	Medium	Acceptable for demo and early beta. Upgrade to paid plan before any public launch with real money. Set Sentry alerts for downtime.
WhatsApp API terms of service	Medium	Use official WhatsApp Business API via approved BSP. Design chama feature so Daraja C2B webhook is sufficient - WhatsApp is enhancement, not dependency.
Play Store financial app policies	Medium	KYC flow satisfies identity verification requirements. No credit scoring submitted externally without consent. Terms of service clearly states fee structure.

Tugetha v2.0

Save Together. Lend to Friends. Achieve Goals.

Built by Sinaps Technology | Nairobi, Kenya | sinapstechnology.tech

synapsetechnology14@gmail.com

Java 21 + Spring Boot 3.x | PostgreSQL + Flyway | Redis | Paystack | Flutter

This document is confidential. For internal use and authorized stakeholder review only.